

CAHIER DES CHARGES

Projet de Formation — JavaScript

QUIZ INTERACTIF

Avec Timer, Scores & Niveaux

Projet	Quiz Interactif JS
Équipe	2 développeurs (vous + votre ami)
Durée	7 jours
Technologies	HTML5 · CSS3 · JavaScript (ES6+)
Objectif	Immersion professionnelle en entreprise
Version	1.0 — Juin 2026

1. Présentation du Projet

1.1 Contexte

Dans le cadre de notre formation accélérée en JavaScript, nous réalisons en binôme un projet web complet sur 7 jours. Ce projet nous permettra d'acquérir les bases solides du développement front-end, indispensables avant notre immersion en entreprise.

Le choix du Quiz Interactif est motivé par sa richesse pédagogique : il couvre tous les concepts fondamentaux du JavaScript moderne tout en produisant un résultat concret, visuel et fonctionnel.

1.2 Objectifs pédagogiques

Ce projet vise à maîtriser les compétences suivantes :

- Manipulation du DOM (Document Object Model)
- Gestion des événements utilisateur
- Logique d'état de l'application (State Management)
- Gestion du temps avec setInterval / setTimeout
- Structures de données : tableaux d'objets, JSON
- Persistance locale avec localStorage
- Architecture propre avec ES6 Modules
- Animations et feedback visuel CSS/JS

1.3 Résultat attendu

Livrable final

Une application web de quiz entièrement fonctionnelle, avec plusieurs catégories de questions, un timer, un système de score, un classement des joueurs, et une interface soignée. Le code sera propre, structuré et commenté.

2. Fonctionnalites Requises

2.1 Fonctionnalites principales (MVP)

Le MVP (Minimum Viable Product) est la version minimale fonctionnelle a livrer obligatoirement.

#	Fonctionnalite	Description	Priorite
F01	Ecran d'accueil	Page de bienvenue avec choix de la categorie et du pseudo	HAUTE
F02	Affichage des questions	Questions affichees une par une avec 4 choix de reponse	HAUTE
F03	Timer par question	Compte a rebours de 30 secondes par question visible	HAUTE
F04	Validation des reponses	Feedback immediat correct/incorrect avec couleur	HAUTE
F05	Calcul du score	Score incremente selon rapidite et justesse	HAUTE
F06	Ecran de resultats	Score final, reponses correctes, bouton rejouer	HAUTE
F07	Sauvegarde localStorage	Historique des scores stocke dans le navigateur	MOYENNE
F08	Tableau des scores	Top 10 des meilleurs scores enregistres	MOYENNE

2.2 Fonctionnalites avancees (bonus)

A implementer si le temps le permet, apres le MVP.

#	Fonctionnalite	Description
B01	Niveaux de difficulte	Facile / Moyen / Difficile avec temps et score differents
B02	Categories multiples	JS, HTML, CSS, Culture generale — au choix
B03	Mode 2 joueurs	Alternance de questions entre deux participants

#	Fonctionnalite	Description
3		
B 0 4	Progression animee	Barre de progression, animations entre questions
B 0 5	Son et effets	Sons pour bonne/mauvaise reponse (Audio API)
B 0 6	Partage du score	Copier le score en texte pour partager

3. Architecture Technique

3.1 Technologies utilisees

Technologie	Version	Usage
HTML5	Standard	Structure des pages et semantique
CSS3	Standard	Mise en forme, animations, responsive design
JavaScript	ES6+	Logique applicative, DOM, evenements
localStorage	Web API	Persistence des scores sans base de donnees
Git	Recommande	Versioning du code et collaboration

3.2 Structure des fichiers

Architecture du projet			
quiz-app/ index.html	— Page principale	styles/ main.css	— Styles globaux
quiz.css	— Styles du quiz	results.css	— Styles des resultats
scripts/ app.js	— Point d'entree, initialisation	quiz.js	— Logique principale du quiz
timer.js	— Gestion du compte a rebours	score.js	— Calcul et sauvegarde des scores
ui.js	— Manipulation du DOM	data/ questions.js	— Banque de questions
README.md	— Documentation du projet		

3.3 Concepts JavaScript couverts

Concept JS	Ou il apparait dans le projet	Jour
Variables let/const, types	Declaration de toutes les donnees du quiz	J1
Fonctions, arrow functions	Toutes les actions du quiz	J1-J2
DOM querySelector / getElementById	Afficher/cacher les ecrans	J2
addEventListener	Clics sur les boutons de reponse	J2
Objets JS et tableaux	Structure des questions (texte + reponses)	J2-J3
Array.forEach / map / filter	Parcourir et filtrer les questions	J3
setInterval / clearInterval	Timer de 30 secondes	J3
Conditions if/else, switch	Verifier la bonne reponse	J2-J3

Concept JS	Où il apparaît dans le projet	Jour
Classes ES6	Classe Quiz, classe Timer, classe Score	J4
localStorage get/set	Sauvegarder le classement	J5
Modules ES6 import/export	Séparer le code en fichiers	J5
JSON.parse / JSON.stringify	Lire et écrire dans localStorage	J5
Promises / async-await (bonus)	Chargement de questions depuis un fichier	J6

4. Plan de Travail — 7 Jours

Chaque journee est planifiee avec des objectifs precis, une repartition des roles, et un livrable concret.

JOUR 1 — Fondations & Structure HTML/CSS

Aspect	Detail
Duree estimee	3 a 4 heures
Theme JS	Variables, types, fonctions de base
Objectif	Avoir un squelette HTML+CSS complet et navigable

Taches du jour

- Creer la structure du dossier projet
- Ecrire index.html avec tous les ecrans (accueil, quiz, resultats)
- Styliser avec CSS : couleurs, polices, disposition generale
- Masquer les ecrans avec CSS (display: none), visible uniquement l'accueil
- Tester la navigation manuelle entre les ecrans

Repartition des roles

Developpeur 1	Developpeur 2
Structure HTML (index.html)	Styles CSS globaux (main.css)
Ecran d'accueil + formulaire pseudo	Ecran de quiz (carte question + reponses)
Ecran des resultats	Ecran du classement

Livrable Jour 1

Un fichier index.html avec 4 ecrans stylises, naviguables manuellement dans le navigateur.

JOUR 2 — DOM & Evenements

Aspect	Detail
Duree estimee	3 a 4 heures
Theme JS	DOM manipulation, addEventListener, conditions
Objectif	Afficher une question et detecter un clic de reponse

Taches du jour

- Creer le fichier scripts/data/questions.js avec 20 questions en tableau d'objets

- Ecrire scripts/ui.js : fonctions showScreen(), hideScreen(), displayQuestion()
- Relier le bouton 'Demarrer' pour afficher l'ecran quiz
- Afficher la premiere question avec ses 4 reponses dynamiquement
- Detecter le clic sur une reponse et afficher si c'est correct ou non

Concept cle : Tableau d'objets

```
const questions = [ { question: "Quelle balise HTML structure le titre principal ?", choices: ["<h1>", "<title>", "<header>", "<main>"], answer: 0, category: "HTML" } ];
```

Livable Jour 2

Clic sur 'Demarrer' -> question apparait avec 4 boutons reponse -> clic -> feedback couleur correct/incorrect.

JOUR 3 — Logique du Quiz & Timer

Aspect	Detail
Duree estimee	4 heures
Theme JS	setInterval, logique d'etat, boucles
Objectif	Quiz complet : enchainement des questions + timer

Taches du jour

- Creer scripts/timer.js : startTimer(), stopTimer(), resetTimer()
- Creer scripts/quiz.js : logique d'etat (question courante, score, fin du quiz)
- Enchaîner les questions automatiquement apres chaque reponse
- Passer a la question suivante automatiquement si le timer expire
- Afficher le numero de question (Ex : Question 3/10)
- Bloquer les boutons apres un clic (eviter le double-clic)

Concept cle : setInterval

```
function startTimer(duration, onTick, onExpire) { let timeLeft = duration; const interval = setInterval(() => { timeLeft--; onTick(timeLeft); if (timeLeft <= 0) { clearInterval(interval); onExpire(); } }, 1000); return interval; }
```

Livable Jour 3

Quiz enchaîne 10 questions avec timer visible. Passage automatique si temps ecoule. Score calcule en temps reel.

JOUR 4 — Classes ES6 & Refactorisation

Aspect	Detail
Duree estimee	3 a 4 heures
Theme JS	Classes, constructeurs, methodes, this
Objectif	Remonter le code en classes propres et reutilisables

Taches du jour

- Creer la classe Quiz avec ses proprietes et methodes
- Creer la classe Timer avec start(), stop(), reset()
- Creer la classe Score avec add(), getTotal(), getRank()
- Remplacer les fonctions eparillees par des appels aux classes
- Tester que tout fonctionne comme avant

Concept cle : Classe ES6

```
class Quiz { constructor(questions) { this.questions = questions; this.currentIndex = 0; this.score = 0; } getCurrentQuestion() { return this.questions[this.currentIndex]; } nextQuestion() { this.currentIndex++; } isFinished() { return this.currentIndex >= this.questions.length; } }
```

Livable Jour 4

Meme application qu'avant, mais le code est reorganise en classes. Revue de code croisee en binome.

JOUR 5 — LocalStorage & Modules ES6

Aspect	Detail
Duree estimee	3 heures
Theme JS	localStorage, JSON, import/export
Objectif	Persister les scores et organiser le code en modules

Taches du jour

- Sauvegarder chaque score dans localStorage apres chaque partie
- Lire et afficher le top 10 des scores sur l'ecran classement
- Transformer chaque fichier .js en module (export/import)
- Relier tous les modules dans app.js (point d'entree)
- Ajouter type="module" dans le script HTML

Concept cle : localStorage

```
// Sauvegarder const scores = JSON.parse(localStorage.getItem('quizScores')) || [];  
scores.push({ name: pseudo, score: total, date: new Date() }); scores.sort((a,b) => b.score - a.score);  
localStorage.setItem('quizScores', JSON.stringify(scores.slice(0,10)));
```

Livrable Jour 5

Scores sauvegardés entre les parties. Tableau des 10 meilleurs scores affiche. Code organisé en modules.

JOUR 6 — Finitions, Bonus & Responsive

Aspect	Detail
Duree estimee	3 a 4 heures
Theme JS	Animations CSS, responsive, fonctionnalites bonus
Objectif	Peaufiner l'experience utilisateur et ajouter du bonus

Taches du jour

- Ajouter une barre de progression (progression de 0 a 100%)
- Animations : transition entre les questions, shake si mauvaise reponse
- Rendre le design responsive (mobile + desktop)
- Ajouter au moins une categorie supplementaire de questions
- Implémenter un ou deux bonus parmi ceux de la section 2.2

Livrable Jour 6

Application visuellement soignée, animée, responsive. Au moins un bonus fonctionnel.

JOUR 7 — Tests, Documentation & Demo

Aspect	Detail
Duree estimee	2 a 3 heures
Theme JS	Qualite du code, commentaires, presentation
Objectif	Livrer un projet propre, documente, presentable

Taches du jour

- Tester tous les cas limites (timer expire, 0 bonnes reponses, mauvais pseudo...)
- Corriger tous les bugs identifiés
- Commenter le code des fonctions et classes clés
- Rédiger le README.md avec : description, screenshot, comment lancer, auteurs
- Préparer une demo de 5 minutes pour présenter le projet
- Revue de code croisée : chacun explique son code à l'autre

Livrable Jour 7

Application livrée, testée, documentée. README complet. Demo préparée. Code sur Git (recommandé).

5. Planning Recapitulatif

Jour	Theme principal	Concept JS cle	Livable
Jour 1	Structure HTML & CSS	Variables, fonctions	4 ecrans HTML stylises
Jour 2	DOM & Evenements	querySelector, addEventListener	Question affichee + feedback
Jour 3	Logique & Timer	setInterval, etat	Quiz 10 questions fonctionnel
Jour 4	Classes ES6	class, constructor, this	Code refactorise en classes
Jour 5	Persistence & Modules	localStorage, import/export	Scores sauvegardes, modules
Jour 6	Finitions & Bonus	Animations CSS, responsive	App polie + bonus
Jour 7	Tests & Demo	Commentaires, README	Projet finalise & documente

6. Repartition des Roles dans l'Equipe

Pour travailler comme des pros, chaque membre a un role clair. La collaboration reste indispensable.

Role	Responsabilites principales	Attribut a
Dev Lead JS	Architecture du code, classes, modules, logique quiz	Developpeur 1
Dev UI/UX	HTML, CSS, animations, responsive, experience utilisateur	Developpeur 2
Responsable Data	Banque de questions, localStorage, JSON	A alterner
Responsable Tests	Tests manuels, correction de bugs, README	A alterner

Bonne pratique

Chaque jour : 15 min de standup au debut (qu'est-ce que j'ai fait hier ? Qu'est-ce que je fais aujourd'hui ? Y a-t-il un blocage ?). Cette pratique est standard dans les equipes Agile en entreprise.

7. Criteres de Reussite

7.1 Criteres obligatoires (MVP)

- Le quiz démarre avec un pseudo saisi
- 10 questions s'enchainent avec 4 choix de reponse chacune
- Un timer de 30 secondes est visible et fonctionnel
- Un feedback couleur indique la bonne/mauvaise reponse
- Le score final est affiche a la fin
- Les scores sont sauvegardes entre les parties
- Un classement des 10 meilleurs scores est visible

7.2 Criteres de qualite du code

- Le code JS est organise en modules separes
- Les classes Quiz, Timer et Score sont implementees
- Les fonctions sont commentees
- Les variables ont des noms explicites (pas de a, b, x...)
- Pas de repetition de code inutile (principe DRY)

7.3 Evaluation finale

Critere	Points	Description
Fonctionnalites MVP	/40	Toutes les fonctionnalites de base fonctionnent
Qualite du code JS	/25	Classes, modules, nommage, lisibilite
Design & UX	/15	Interface soignee, responsive, animations
Documentation	/10	README, commentaires dans le code
Fonctionnalites Bonus	/10	Pour chaque bonus fonctionnel implementé
TOTAL	/100	Score de reussite minimum : 60/100

8. Ressources et References

8.1 Documentation officielle

- MDN Web Docs (JS) : developer.mozilla.org/fr/docs/Web/JavaScript
- MDN DOM : developer.mozilla.org/fr/docs/Web/API/Document_Object_Model
- MDN localStorage : developer.mozilla.org/fr/docs/Web/API/Window/localStorage

8.2 Outils recommandes

- Editeur : Visual Studio Code (gratuit)
 - Extensions : Prettier (formatage), ESLint (qualite), Live Server (reload auto)
- Versioning : Git + GitHub (compte gratuit)
 - Commandes essentielles : git init, git add, git commit, git push
- Navigateur : Chrome ou Firefox avec DevTools ouvertes (F12)

8.3 Pratiques professionnelles a adopter

- Commits Git reguliers avec messages clairs (ex: 'feat: ajout du timer')
- Revue de code quotidienne en binome
- Nommage en anglais pour les variables et fonctions
- Indentation coherente (2 ou 4 espaces, a definir ensemble)
- Un fichier = une responsabilite (separation des preoccupations)

9. Glossaire Technique

Terme	Definition
DOM	Document Object Model — representation de la page HTML en objets JS manipulables
Event Listener	Fonction qui 'ecoute' un evenement (clic, frappe...) et reagit
setInterval	Fonction JS qui repete une action a intervalle regulier (ex: toutes les secondes)
localStorage	Stockage de donnees dans le navigateur, persistant meme apres fermeture
ES6 Modules	Systeme d'organisation du code JS en fichiers separes avec import/export
MVP	Minimum Viable Product — version minimale fonctionnelle du produit
CRUD	Create, Read, Update, Delete — les 4 operations de base sur des donnees
State	Etat de l'application a un instant T (question courante, score, temps restant...)
Refactoring	Reorganiser/ameliorer le code sans changer son comportement
DRY	Don't Repeat Yourself — principe de non-repetition du code

Bonne formation et bonne immersion en entreprise !

Quiz Interactif JS — Cahier des Charges v1.0 — Juin 2026